

L021

Rapport final

Personne en charge : Lise Casanova Kinkela et Emma Van
Goethem

HARMONIES

Léonie Leroux, Estelle Pingard, Lise Casanova Kinkela, Basma Fengoura
et Emma Van Goethem

Introduction	2
Partie 1 - La Synthèse	3
Ce qui a été implémenté	3
Ce qui n'a pas été implémenté	5
Partie 2 - L'Architecture	6
La couche modèle - les ressources du jeu	6
La couche modèle - les cartes	7
La couche logique - la gestion de partie	8
La couche calcul du score	9
La couche interface graphique Qt	9
Évolutivité de l'application	10
L'UML	11
Partie 4 - Le Planning effectif	13
Semaine 1: Semaine du 17 mars	13
Semaine 2 - Semaine du 23 mars (Livrable 1)	13
Semaines 3 et 4 - Début avril	13
Semaines 5 et 6 - Mi-avril	14
Semaine 7 - Semaine du 27 avril (Livrable 2)	14
Semaines 8 et 9 - Début mai	14
Semaines 10 et 11 - Mi-mai	15
Semaine 12 - Semaine du 26 mai (Livrable 3)	15
Semaines 13 et 14 - Début juin	15
Semaine 15 - Semaine du 13 juin (Livrable final)	16
Partie 5 - Les Contributions	17
Tableau récapitulatif	17
Léonie Leroux	18
Lise Casanova Kinkela	18
Estelle Pingard	19
Emma Van Goethem	19
Basma Fengoura	20
Conclusion	21

Introduction

Le but de ce projet est de concevoir et développer une application permettant de jouer au jeu de société Harmonies, créé par Johan Benvenuto et édité par Libellud. Dans ce jeu de pose de jetons, les joueurs s'affrontent en construisant des territoires à l'aide de différents éléments (montagnes, bâtiments, arbres, eau, champs) afin de pousser des animaux à venir s'installer sur les habitats qui leur conviennent.

Ce jeu se joue normalement de 2 à 4 joueurs mais propose également un mode solo. De manière optionnelle, il est aussi possible d'utiliser les cartes Esprit qui permettent de corser la partie.

Ce projet s'est étalé tout au long du semestre. Il a demandé un travail régulier ainsi qu'une coopération accrue entre les différents membres de notre groupe, notamment pour faire la transition entre le code qui tournait au début dans la console de commande et l'interface graphique finale développée sur Qt.

Ainsi, dans ce rapport final, nous souhaitons établir :

- Une synthèse de ce que permet notre application, où l'on précisera, parmi les opérations attendues, celles qui ont été implémentées ou non.
- Une description précise de notre architecture et les raisons pour lesquelles celle-ci permet facilement des évolutions.
- Les diagrammes UML qui permettront une bonne description de nos sous-parties logiques et graphiques.
- Une description du planning effectif de notre projet au fil des semaines.
- Une description de la contribution personnelle de chacun des membres du groupe sur les différents livrables, ainsi qu'une évaluation en pourcentage et en heures sur l'ensemble du projet.

Ce rapport final permettra de comprendre l'ensemble du travail que nous avons fourni tout au long du semestre pour donner vie à ce jeu.

Partie 1 - La Synthèse

L'application développée au cours de ce semestre permet de jouer au jeu de société Harmonies en C++, avec une interface graphique réalisée sous Qt. Cette synthèse présente les fonctionnalités attendues par le sujet et précise, pour chacune d'elles, si elle a été implémentée ou non.

Ce qui a été implémenté

Le cœur du jeu, le mode console et le mode graphique. L'application est jouable dans les deux modes demandés. Le mode console, développé en premier, permet de dérouler une partie complète : initialisation, tours de jeu, calcul des scores et annonce du gagnant. Le mode graphique Qt, développé dans un second temps, reprend l'intégralité de cette logique et l'expose au travers d'une interface interactive.

La configuration d'une partie. Au lancement, l'application permet de paramétrer la partie. En mode console, la classe **CreationPartie** gère la saisie du nombre de joueurs (de 1 à 4), leurs noms, le type de plateau (A ou B) et le mode de jeu (avec ou sans cartes Esprit). En mode graphique, ces mêmes paramètres sont saisis via la classe **SetupDialog**, qui propose des champs dynamiques s'adaptant au nombre de joueurs choisis. La fenêtre d'accueil **MainWindow** détecte automatiquement la présence d'une sauvegarde et propose de la reprendre ou de l'abandonner.

La gestion des jetons et du plateau central. La pioche contient les 120 jetons du jeu répartis en six couleurs, mélangés aléatoirement. Le plateau central (**PlateauC**) dispose de 5 cases (3 en mode solo), chacune contenant 3 jetons. À chaque tour, le joueur choisit une case et récupère ses 3 jetons. Les cases vides sont automatiquement remplies depuis la pioche. En mode graphique, le widget **PlateauCentralWidget** affiche les jetons de chaque case sous forme de ronds colorés cliquables.

Le plateau joueur et le placement des jetons. Chaque joueur dispose d'un plateau personnel (**PlateauJ**) dont la forme dépend du type choisi : le plateau A est en quinconce avec des lignes de 5 et 4 cases alternées, le plateau B est rectangulaire (4 lignes × 7 colonnes). Les jetons sont empilables selon des règles strictes vérifiées par la classe **Verificateur**: les jetons bleus et jaunes ne se posent que sur une case vide, les jetons marrons peuvent s'empiler jusqu'à deux étages, les jetons gris jusqu'à trois, les jetons rouges sur le vide, le gris ou le marron, et les jetons verts uniquement au sommet d'une pile marron. En mode graphique, le widget **PlateauJoueurWidget** affiche le plateau sous forme d'hexagones dessinés avec QPainter, avec un rendu visuel des couleurs et des empilements.

Les cartes Animal et les cubes. La pioche de cartes Animal (**PiocheAnimaux**) contient les 32 cartes du jeu, créées et initialisées par la fonction **initialiserCartesAnimaux**. Cinq cartes sont visibles à tout moment (trois en mode solo). Chaque joueur peut en prendre une par tour, dans la limite de 4 cartes actives simultanées. Le placement d'un cube Animal sur le plateau nécessite que le motif de la carte soit présent sur le plateau du joueur, dans n'importe quelle orientation : les six rotations possibles sont gérées par la classe **AnimalMotif** via un système de coordonnées hexagonales cubiques (**HexCoord**). Le calcul du score des cartes Animal est assuré par **CalculScore::calculerScoreAnimaux**.

Les cartes Esprit. En mode avec cartes Esprit, la pioche **PiocheEsprits** contient les 10 cartes du jeu. En début de partie, deux cartes sont distribuées à chaque joueur, qui en choisit une. Tant que le cube Esprit n'est pas posé, la carte compte dans la limite des 4 cartes actives. Les effets des cartes Esprit modifient le calcul du score des paysages selon des règles spécifiques à chaque carte, gérées par la structure **EspritScoring**.

Le calcul des scores. La classe **ScorePaysage** calcule les points liés aux paysages : arbres (selon la hauteur de la pile marron + vert), montagnes (selon la hauteur et l'adjacence), champs (groupes de jetons jaunes adjacents), bâtiments (jetons rouges entourés d'au moins 3 couleurs différentes) et eau (rivière la plus longue pour le plateau A, nombre d'îles pour le plateau B). La classe **CalculScore** agrège les scores des paysages, des cartes Animal et des cartes Esprit pour produire le score total de chaque joueur. En mode solo, le score est converti en Soleils selon le barème du jeu (voir livrable 1 pour les règles spécifiques au mode solo).

La sauvegarde de partie. La classe **SauvegardePartie** permet d'enregistrer l'état complet d'une partie dans un fichier texte et de la recharger ultérieurement. La sauvegarde inclut le mode de jeu, le type de plateau, l'état de la pioche, le plateau central, les plateaux joueurs case par case, les cartes Animal et Esprit de chaque joueur ainsi que les cubes posés. Elle ne peut être effectuée qu'en fin de tour. La sauvegarde est automatiquement supprimée à la fin d'une partie.

Le mode solo. Le mode solo est pleinement fonctionnel. Le plateau central ne comporte que 3 cases. À chaque fin de tour, les jetons des cases non choisies sont défaussés (non remis dans le sac) et le plateau central est entièrement rechargé. Si le joueur ne prend pas de carte Animal durant son tour, il peut en défausser une des cartes visibles pour la remplacer par une nouvelle depuis la pioche.

L'interface graphique Qt. L'interface graphique est composée de plusieurs fenêtres et widgets interconnectés via le mécanisme de signaux et slots de Qt : **MainWindow** (menu principal), **SetupDialog** (configuration), **GameWindow** (fenêtre principale de jeu), **PlateauCentralWidget**, **PlateauJoueurWidget**, **CartesAnimalWidget**, **ScoreWidget** et **ReglesDialog** (règles par onglets). La **GameWindow** orchestre l'ensemble du déroulement du tour et traduit chaque action utilisateur en appel au moteur de jeu.

Ce qui n'a pas été implémenté

Une fonctionnalité que nous aurions souhaité implémenter est un **système d'aide à la décision**. L'idée aurait été de proposer au joueur, à chaque tour, une suggestion de placement optimal en analysant l'état courant du plateau et les cartes Animal actives. Ce système aurait pu mettre en évidence les cases où poser un jeton permettrait de compléter un motif de carte Animal, ou encore signaler les placements qui maximisent le score paysage en fin de partie.

Notre architecture s'y prêterait bien : la classe **AnalysePlateau** fournit déjà les primitives nécessaires pour analyser le plateau (voisins, groupes de couleurs, cases vides), et **AnimalMotif::corrMotifPlateau** permet de vérifier si un motif est présent à une position donnée. Il aurait suffi de créer une classe **AideDecision** s'appuyant sur ces deux modules pour parcourir les placements possibles et évaluer leur intérêt. En mode graphique, le résultat aurait pu être affiché directement sur le **PlateauJoueurWidget** en surlignant les cases suggérées.

Partie 2 - L'Architecture

Notre application repose sur une séparation claire entre trois grandes couches : le modèle de données du jeu, la logique de gestion de partie, et l'interface graphique. Ce découpage, inspiré du patron MVC (Modèle-Vue-Contrôleur), a été adopté dès le livrable 2 pour permettre à chaque membre du groupe de travailler indépendamment sur sa partie, avant de tout intégrer. Il a également facilité la transition du mode console vers le mode graphique Qt, qui constituait l'un des défis majeurs du projet.

La couche modèle - les ressources du jeu

La base du projet repose sur le fichier **RessourcePJ.h**, qui regroupe les structures fondamentales manipulées tout au long d'une partie.

La classe **Jeton** est la brique élémentaire du jeu. Elle encapsule simplement une **CouleurJeton** (enum à six valeurs : BLEU, GRIS, MARRON, VERT, JAUNE, ROUGE). Ce choix de séparer le jeton de sa couleur, plutôt que de manipuler directement l'enum, rend le code plus lisible et permet d'ajouter facilement de nouveaux comportements au jeton sans toucher au reste.

La classe **Pioche** gère l'ensemble des 120 jetons de la partie. Elle alloue dynamiquement un tableau de **Jeton** au moment de sa construction, les mélange aléatoirement, et expose une méthode **piocher()** qui retourne un pointeur vers le jeton suivant. Le choix de retourner un pointeur vers le jeton dans le tableau plutôt qu'une copie est délibéré : cela évite la duplication des jetons en mémoire et garantit que chaque jeton n'existe qu'en un seul exemplaire, ce qui est cohérent avec la réalité physique du jeu.

La classe **Place** représente une case du plateau joueur. Elle contient trois pointeurs vers des **Jeton** (les trois étages possibles d'empilement), initialisés à `nullptr`. L'opérateur `<<` permet d'ajouter un jeton à l'étage disponible suivant de façon intuitive. Ce choix d'une structure à trois pointeurs fixes plutôt qu'un tableau dynamique est justifié par le fait que le jeu n'autorise jamais plus de trois étages : cela simplifie la gestion mémoire et rend le code plus direct.

La classe **PlateauJ** représente le plateau personnel d'un joueur. Elle gère un tableau bidimensionnel de "Place*", dont la forme dépend du type choisi (A ou B). Le plateau A est en quinconce c'est à dire que les lignes paires ont 5 cases et les lignes impaires en ont 4 ce qui est géré par la méthode **getNbColonneLigne(int ligne)**. Le plateau B est rectangulaire (4 × 7). Ce choix de gérer la forme du plateau dans la classe elle-même, plutôt que dans une classe externe, évite de disperser cette logique et rend les accès aux cases cohérents depuis n'importe quel module. **PlateauJ** gère également un tableau parallèle de **CubeAnimal**** pour savoir si un cube Animal est posé sur chaque case, sans polluer la structure **Place** avec cette information.

La classe **PlateauC** représente le plateau central. Elle contient un tableau plat de pointeurs vers des Jeton, organisé en cases de trois jetons chacune. Ses méthodes **ajouterC** et **retirerC** permettent respectivement de remplir une case depuis la pioche et de transférer les jetons d'une case vers le plateau d'un joueur.

La classe **Verificateur** est séparée des plateaux et expose uniquement des méthodes statiques. Ce choix est important : la vérification des règles de placement n'a pas d'état propre, et la rendre statique permet de l'appeler depuis n'importe quel contexte sans instancier d'objet inutile. **calculerHauteur** détermine le nombre d'étages occupés sur une case, et **estPossible** vérifie si un jeton d'une couleur donnée peut légalement y être posé.

La couche modèle - les cartes

La gestion des cartes repose sur une hiérarchie d'héritage construite autour d'une classe de base **Carte**. Ce choix a été fait lors du livrable 3, quand il est apparu que les cartes Animal et les cartes Esprit partageaient de nombreux attributs communs : un nom, un motif (**AnimalMotif**), un nombre maximum de placements et un compteur de cubes déjà posés.

CarteAnimal hérite de **Carte** et ajoute un tableau de scores (**tableauScore**) qui associe un nombre de points à chaque niveau de placement. **CarteEsprit** hérite également de **Carte** et ajoute une structure **EspritScoring** qui décrit les effets de la carte sur le calcul du score des paysages, ainsi qu'un booléen choisie indiquant si le joueur a retenu cette carte en début de partie.

La classe **AnimalMotif** est au cœur de la mécanique de placement des animaux. Elle représente un motif sous forme d'un ensemble de **TuileMotif**, chacune décrivant la couleur et l'étage attendus à une position hexagonale donnée (exprimée en **HexCoord**). La méthode **rotate(int n)** génère le motif pivoté de $n \times 60^\circ$ sans modifier l'original, et la méthode template **corrMotifPlateau** vérifie si le motif correspond au plateau à partir d'une origine et d'une rotation données. Le choix d'une méthode template ici est justifié par la volonté de garder **AnimalMotif** indépendante de **PlateauJ** : la méthode reçoit des fonctions lambda pour accéder aux cases du plateau, ce qui évite que les deux classes soient directement liées.

Le système de coordonnées hexagonales **HexCoord** mérite une attention particulière. Plutôt que de travailler directement en coordonnées offset (ligne/colonne), les rotations sont effectuées en coordonnées cubiques (q, r, s), ce qui simplifie considérablement les calculs mathématiques. Les méthodes **versCube()**, **deCube()** et **pivote60()** assurent les conversions nécessaires. Ce choix technique, introduit par Estelle au livrable 2, a été déterminant pour résoudre la difficulté anticipée dès le livrable 1 concernant la gestion des rotations des motifs.

Les pioches **PiocheAnimaux** et **PiocheEsprits** encapsulent la gestion des cartes cachées et visibles. **PiocheAnimaux** maintient un tableau fixe de cinq pointeurs vers les cartes visibles, avec des méthodes dédiées pour prendre une carte visible, la remplacer depuis la pioche cachée et gérer le mode solo. L'interdiction de copie (constructeur et opérateur

d'affectation supprimés) évite les doublons de pointeurs, qui auraient pu provoquer des suppressions multiples en mémoire.

La couche logique - la gestion de partie

La logique de jeu est organisée autour de trois classes principales : **GestionPartie**, **GestionTours** et **Joueur**.

La classe **Joueur** hérite de **PlateauJ**. Ce choix d'héritage, discuté lors du livrable 2, s'explique par le fait qu'un joueur est conceptuellement indissociable de son plateau : il n'existe pas de joueur sans plateau, et toutes les opérations du tour (poser un jeton, vérifier un motif, poser un cube) s'effectuent directement sur le plateau du joueur. **Joueur** ajoute à **PlateauJ** le nom du joueur, un tableau de pointeurs vers ses **CarteAnimal** actives (limité à 4) et un vecteur de **CarteEsprit**. L'interdiction de copie est maintenue pour éviter les problèmes de mémoire liés au tableau dynamique de cartes.

La classe **GestionTours** orchestre le déroulement d'un tour pour un joueur donné. Elle reçoit par référence la pioche, le plateau central, le joueur courant et la pioche de cartes Animal, ce qui garantit qu'elle travaille toujours sur les objets réels sans les dupliquer. Un tableau de booléens **effectu[3]** suit les trois actions possibles du tour (prise de jetons, prise de carte Animal, pose de cube). Ce choix permet de vérifier à tout moment si le tour est complet avant de le valider.

La classe **GestionPartie** est le point central de la logique de jeu. Elle possède l'ensemble des ressources partagées (pioche de jetons, plateau central, pioches de cartes) et le vecteur de pointeurs vers les joueurs. Elle expose à l'interface graphique Qt un ensemble de méthodes d'accès (**getJoueurCourant()**, **getPlateauCentral()**, **getPiocheAnimaux()**, etc.) qui permettent à **GameWindow** d'interroger le moteur sans accéder directement à ses attributs privés, préservant ainsi l'encapsulation.

La méthode **passerAuJoueurSuivant()** gère la rotation des joueurs et l'incrémentation du numéro de tour. **GestionPartie** délègue la sauvegarde et le chargement à ses propres méthodes **sauvegarderPartie** et **chargerPartie**, qui s'appuient sur les capacités de sérialisation ajoutées à chaque classe du modèle.

La couche calcul du score

Le calcul du score est découpé en deux classes aux responsabilités distinctes. **ScorePaysage** analyse le plateau joueur et calcule les points liés à chaque type de paysage de façon indépendante. Chaque type de paysage dispose de sa propre méthode statique (**calculerScoreArbres**, **calculerScoreMontagnes**, etc.), ce qui rend le code facile à tester et à modifier unitairement. Pour le calcul de l'eau, deux méthodes privées distinctes (**calculerScoreRiviere** et **calculerScoreIlles**) gèrent les deux faces du plateau. **ScorePaysage** s'appuie sur **AnalysePlateau**, dont les méthodes statiques (**getVoisins**, **trouverGroupesCouleur**, **compterCouleursVoisinesDifferentes**) fournissent les primitives d'analyse du plateau sans dupliquer cette logique dans chaque calcul.

CalculScore agrège l'ensemble des scores et expose plusieurs surcharges de **calculerScoreTotal** selon le contexte (avec ou sans cartes Esprit, avec ou sans mode solo). En mode solo, elle prend en charge la conversion du score en Soleils via **calculerSoleils**, en tenant compte du type de plateau et de la présence ou non de cartes Esprit. Ce découpage en deux classes distinctes, plutôt qu'une seule classe monolithique, a été adopté suite aux difficultés d'intégration rencontrées lors du livrable 3, où le calcul du score dépendait de modules encore en cours de développement.

La couche interface graphique Qt

L'interface graphique repose sur le patron de conception signaux/slots de Qt, qui joue ici le rôle de médiateur entre les widgets et le moteur de jeu. Aucun widget ne modifie directement l'état du jeu : chaque action utilisateur émet un signal, que **GameWindow** intercepte pour appeler la méthode correspondante du moteur, puis rafraîchit l'affichage.

MainWindow est la fenêtre d'entrée de l'application. Elle détecte la présence d'une sauvegarde au démarrage et oriente l'utilisateur vers **SetupDialog** (nouvelle partie) ou directement vers **GameWindow** (reprise). **SetupDialog** remplace la classe **CreationPartie** console et produit un **ParametresPartie**, structure de données simple transmise à **GestionPartie** pour initialiser la partie.

GameWindow est la fenêtre centrale. Elle instancie et orchestre les quatre widgets principaux : **PlateauCentralWidget**, **PlateauJoueurWidget**, **CartesAnimalWidget** et **ScoreWidget**. Elle maintient l'état du tour courant (jetons en attente, carte prise, case sélectionnée) et appelle **GestionPartie** à chaque étape pour valider les actions. En fin de tour, elle appelle **passerAuJoueurSuivant()** et rafraîchit l'ensemble de l'affichage.

PlateauJoueurWidget mérite une mention particulière : les cases du plateau sont dessinées comme de véritables hexagones à l'aide de QPainter, en surchargeant **paintEvent**. La méthode **hitButton** est également surchargée pour que les clics ne soient détectés qu'à l'intérieur de l'hexagone et non dans son rectangle englobant, ce qui donne un rendu fidèle à la forme réelle du plateau.

Des captures d'écran de l'interface graphique finale sont disponibles en annexe.

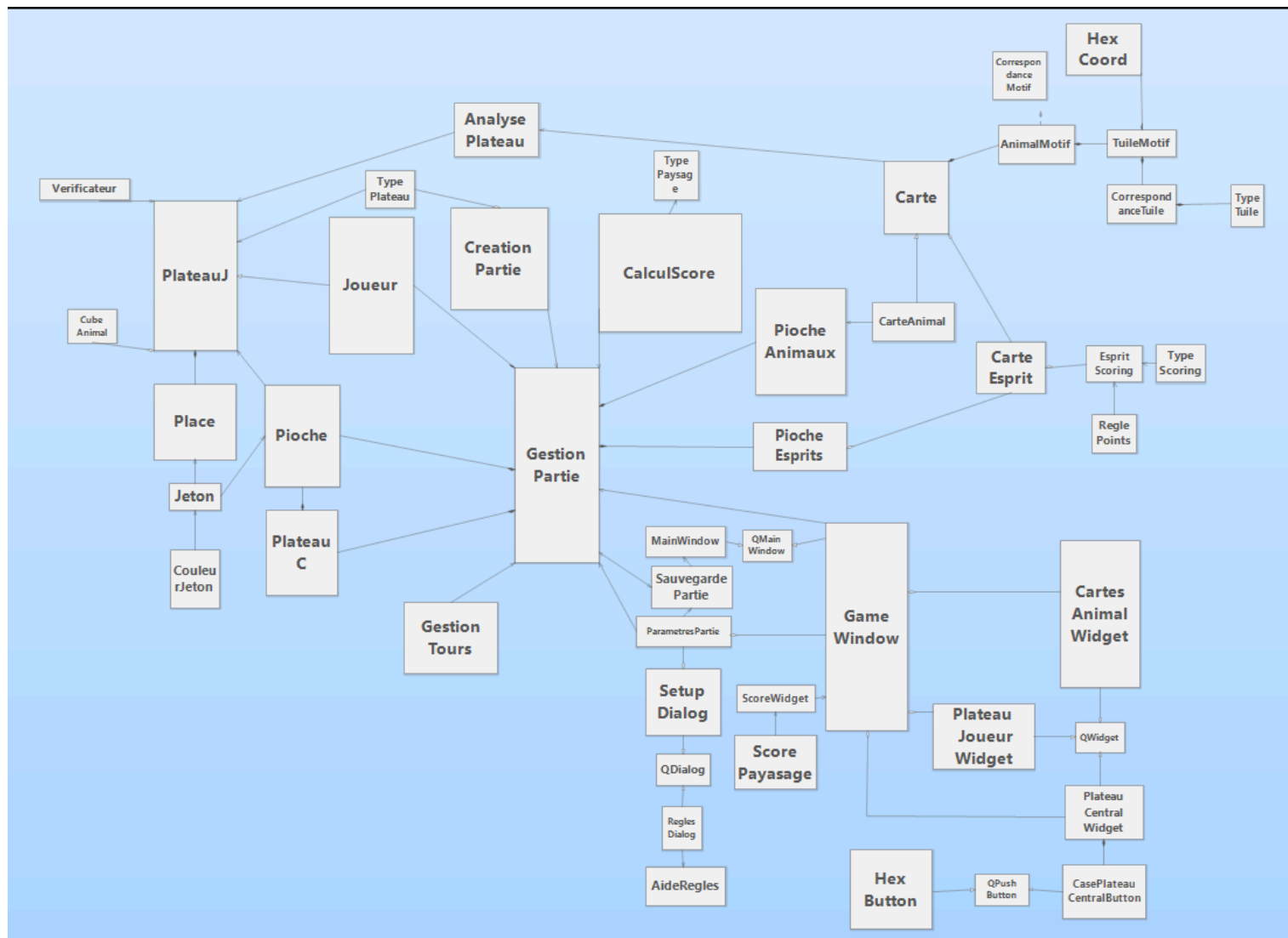
Évolutivité de l'application

Le sujet demande que l'architecture permette d'intégrer de nouveaux composants sans impacter le reste du programme. Nous avons conçu notre application de façon à faciliter ces extensions. Voici comment les deux évolutions citées dans le sujet auraient pu être menées à bien.

Ajouter de nouvelles cartes Animal. La hiérarchie de classes repose sur une classe de base **Carte**, dont héritent **CarteAnimal** et **CarteEsprit**. Chaque carte Animal est définie par un nom, un motif (**AnimalMotif**) et un tableau de scores. Ajouter une nouvelle carte revient simplement à créer une nouvelle instance de **CarteAnimal** avec le motif souhaité, exprimé sous forme de **TuileMotif** avec leurs coordonnées hexagonales et leurs couleurs attendues, puis à l'injecter dans la pioche via **initialiserCartesAnimaux**. Aucune autre classe n'est à modifier : le moteur de jeu, la vérification de placement et le calcul du score fonctionneraient immédiatement avec cette nouvelle carte, car toute la logique est générique et repose sur le motif plutôt que sur le nom ou l'identité de la carte.

Ajouter de nouvelles façons de compter les points. Le calcul des scores est découpé en couches indépendantes. La classe **ScorePaysage** regroupe un ensemble de méthodes statiques, une par type de paysage. Ajouter une nouvelle règle de score reviendrait à ajouter une méthode statique dans cette classe (par exemple **calculerScoreNouvelleMecanique**) et à l'appeler depuis **calculerScorePaysage**. La classe **CalculScore**, qui agrège les différents scores, n'aurait à modifier qu'une seule ligne pour intégrer ce nouveau calcul. De même, pour les cartes Esprit, la structure **EspritScoring** et ses **ReglePoints** permettent de définir n'importe quelle tranche de points sans modifier le code de calcul lui-même : il suffit de renseigner les nouvelles règles dans la carte concernée.

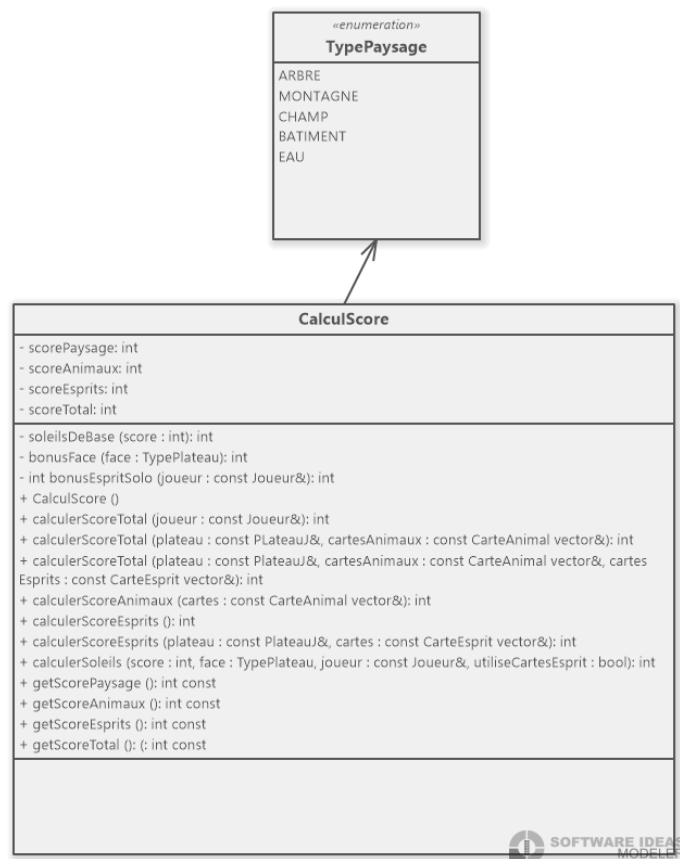
L'UML



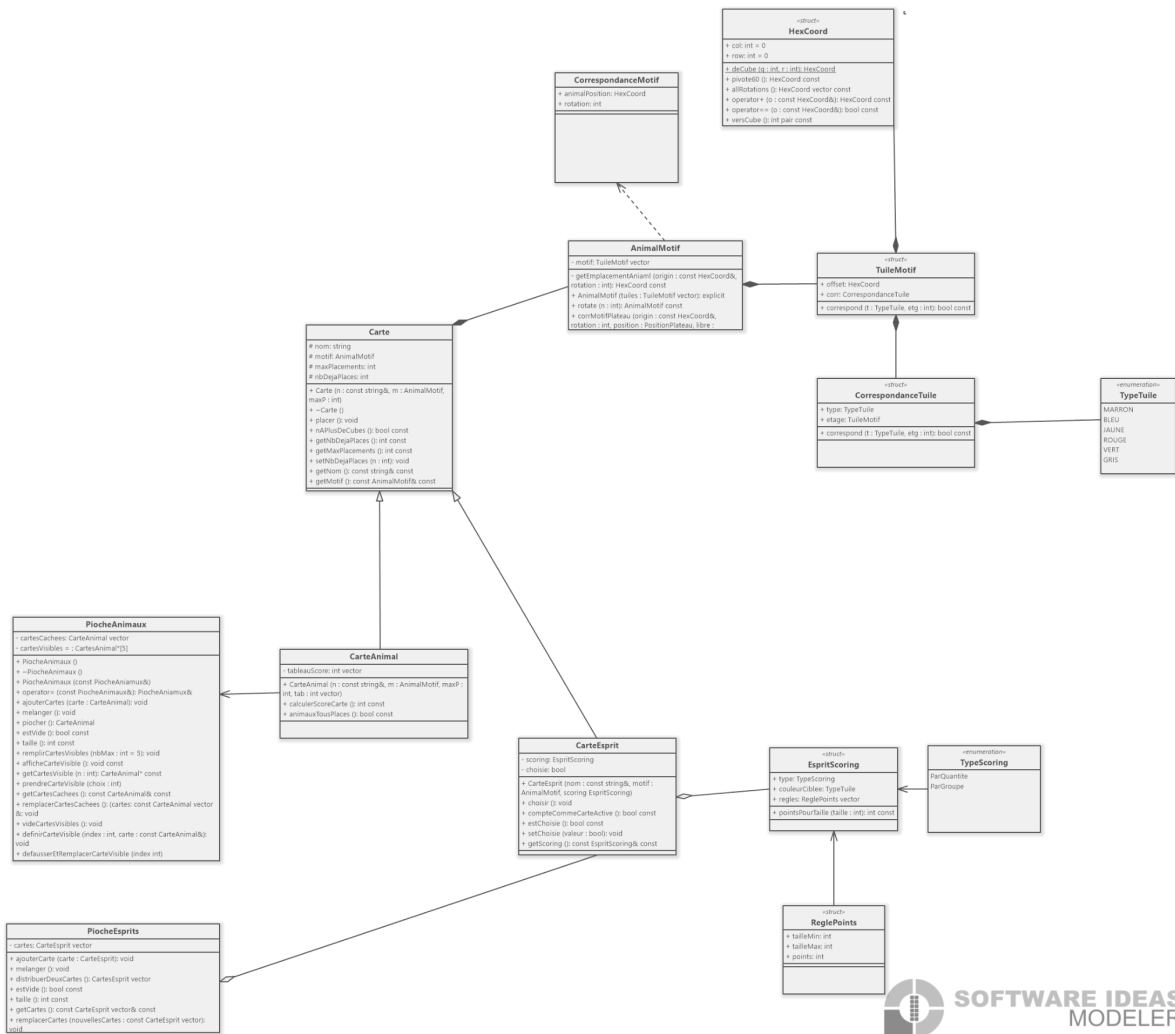
Le **premier diagramme** offre une vue globale de l'ensemble des classes et de leurs relations. On y distingue clairement les trois grandes couches de l'application : à gauche, les classes du modèle de données (**Jeton**, **Place**, **Pioche**, **PlateauJ**, **PlateauC**, **CubeAnimal**) ; au centre, les classes de logique de jeu (**GestionPartie**, **GestionTours**, **Joueur**, **CreationPartie**, **CalculScore**) ; à droite, les classes liées aux cartes et aux motifs (**Carte**, **CarteAnimal**, **CarteEsprit**, **AnimalMotif**, **HexCoord**, **TuileMotif**). L'interface graphique Qt (**MainWindow**, **GameWindow**, **SetupDialog** et les différents widgets) apparaît en bas au centre, reliée au moteur de jeu via **GestionPartie**. Ce découpage visuel reflète fidèlement la séparation MVC adoptée dès le livrable 2.



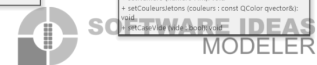
12



Le **troisième diagramme** est consacré à la classe **CalculScore** et à l'énumération **TypePaysage**. Il illustre la séparation entre le calcul par type de paysage (délégué à **ScorePaysage**) et l'agrégation finale des scores (assurée par **CalculScore**). Les différentes surcharges de **calculerScoreTotal** reflètent les contextes de jeu possibles : avec ou sans cartes Esprit, avec ou sans mode solo.



Le **quatrième diagramme** présente la hiérarchie des cartes et le système de coordonnées hexagonales. La classe de base **Carte** est héritée par **CarteAnimal** et **CarteEsprit**, ce qui permet de partager les attributs communs (nom, motif, compteur de placements). La classe **AnimalMotif** est au centre du diagramme : elle agrège un ensemble de **TuileMotif**, chacune décrite par une position en **HexCoord** et un type de tuile (**TypeTuile**). La structure **EspritScoring** et ses **ReglePoints** apparaissent comme des compositions de **CarteEsprit**, illustrant la façon dont les effets de scoring des cartes Esprit sont définis de manière déclarative sans modifier le moteur de calcul. Les pioches **PiocheAnimaux** et **PiocheEsprits** encapsulent la gestion des cartes visibles et cachées.



15



Le **sixième diagramme** présente en détail les classes **GestionPartie** et **GestionTours**, dont les méthodes publiques constituent l'API exposée à l'interface graphique. On y voit notamment l'ensemble des accesseurs de **GestionPartie** (**getJoueurCourant**, **getPlateauCentral**, **getPiocheAnimaux**, etc.), qui permettent à **GameWindow** d'interroger le moteur sans accéder directement à ses attributs privés, préservant ainsi l'encapsulation. La classe **GestionTours** y apparaît avec son tableau de booléens `effectu[3]`, qui suit l'avancement des trois actions d'un tour.

Partie 4 - Le Planning effectif

Le projet s'est étalé sur l'ensemble du semestre de printemps 2026, de la semaine du 23 mars à la semaine du 13 juin, soit environ 12 semaines de travail actif.

Dans cette partie, nous allons présenter semaine par semaine les tâches effectuées ainsi que leur avancement.

Semaine 1: Semaine du 17 mars

- Découverte et prise en main du jeu Harmonies - 100%
- Lecture des règles complètes - 100%
- Mise en place du dépôt Git commun - 100%

Cette première semaine a été consacrée exclusivement à la découverte du jeu. L'ensemble du groupe a joué plusieurs parties d'Harmonies afin de bien comprendre les mécanismes avant d'entamer la moindre réflexion sur le code.

Semaine 2 - Semaine du 23 mars (*Livrable 1*)

- Analyse des règles et identification des entités du jeu - 100%
- Premier modèle UML conceptuel - 100%
- Répartition des tâches entre les membres - 100%
- Établissement du planning macro - 100%

Le livrable 1 a été rendu cette semaine. Il constitue une première analyse conceptuelle du jeu : identification des grandes entités (plateau, jetons, cartes, joueurs), ébauche d'un premier UML et définition d'un planning pour la suite du semestre. Lors du premier livrable, nous n'étions que quatre, la répartition s'est donc adaptée par la suite à l'arrivée de Basma dans le groupe.

Semaines 3 et 4 - Début avril

- Création de l'enum **CouleurJeton** et de la classe **Jeton** - 100%
- Création de la classe **Pioche** - 70%
- Création de la classe **Place** - 100%
- Création de la classe **PlateauJ** - 40%
- Création de la classe **PlateauC** - 50%
- Création de la classe **CreationPartie** - 40%

Les premières classes fondamentales du jeu ont été développées. Chaque membre a travaillé sur sa partie de façon relativement indépendante. Les modules ne sont pas encore reliés entre eux, mais les bases sont posées.

Semaines 5 et 6 - Mi-avril

- Finalisation de **PlateauJ** et **PlateauC** - 80%
- Finalisation de **Pioche** - 80%
- Implémentation de **HexCoord** et **AnimalMotif** - 100%
- Début de **CarteAnimal** et **PiocheAnimaux** - 40%
- Création de **VérificationPlace** - 60%
- Diagramme UML global mis à jour - 60%

Les coordonnées hexagonales ont été implémentées, ce qui a permis de poser les bases de la gestion des motifs des cartes Animal. Le diagramme UML a été mis à jour pour refléter les nouvelles classes et leurs relations. Les modules restent encore largement indépendants.

Semaine 7 - Semaine du 27 avril (*Livrable 2*)

- Mise en place du **main.cpp** commun avec menu de test - 100%
- **CreationPartie** fonctionnelle en console - 50%
- Remplissage du plateau central et de la pioche - 50%
- Finalisation de **VérificationPlace** - 100%

Le livrable 2 a été rendu cette semaine. Il présente l'état d'avancement de chaque module et met en évidence les premières difficultés d'intégration. L'arrivée de Basma dans le groupe a nécessité une redistribution des rôles : elle prend en charge le calcul du score, l'analyse du plateau..

Semaines 8 et 9 - Début mai

- Finalisation de **CarteAnimal**, **CarteEsprit**, **PiocheAnimaux**, **PiocheEsprits** - 100%
- Création des 32 cartes Animal et des 10 cartes Esprit - 100%
- Création de la classe **Joueur** - 100%
- Création de **GestionsTours** - 90%
- **AnalysePlateau** et **Score Paysage** - 80%
- **CalculScore** - 70%

Cette période a été particulièrement productive. Les cartes Animal et Esprit ont été entièrement créées et intégrées dans leurs pioches respectives. La classe **Joueur** a été développée en héritant de **PlateauJ**, et la gestion des tours a commencé à prendre forme. Le calcul du score avance mais dépend encore de modules en cours de finalisation.

Semaines 10 et 11 - Mi-mai

- Correction globale et stabilisation du code - 100%
- Fusion des modules développés séparément - 100%
- Correction des crashes mémoire dans **PlateauJ** - 100%
- Finalisation de **GestionPartie**, **CalculScore** - 100%
- Version console jouable - 90%

Cette période a été la plus complexe du projet. L'intégration des modules développés en parallèle a révélé de nombreuses incompatibilités : headers manquants, types incompatibles, définitions dupliquées, crashes mémoire. Un travail important de stabilisation a été nécessaire avant de pouvoir tester une partie complète en console. À l'issue de cette période, le jeu est jouable en mode console dans ses grandes lignes.

Semaine 12 - Semaine du 26 mai (*Livrable 3*)

- Affichage des scores finaux et annonce du gagnant - 100%
- Amélioration et finalisation du menu console - 100%
- Début de l'interface graphique Qt - 20%
- Version console complète et testée - 100%

Le livrable 3 a été rendu cette semaine. La version console est désormais fonctionnelle et jouable en mode solo et multijoueur. L'interface graphique Qt est en cours de développement.

Semaines 13 et 14 - Début juin

- **MainWindow, SetupDialog, PlateauCentralWidget, CartesAnimalWidget, ScoreWidget, ReglesDialog** - 100%
- **GameWindow** et orchestration du tour - 100%
- **PlateauJoueurWidget** avec rendu hexagonal QPainter - 100%
- Système de sauvegarde complet - 100%
- Réimplémentation du mode solo - 100%
- Tests finaux et corrections d'intégration Qt - 100%

La dernière ligne droite a été consacrée au développement de l'interface graphique Qt dans son intégralité, ainsi qu'à la finalisation du système de sauvegarde et du mode solo. Des corrections d'intégration ont été nécessaires pour relier le moteur console à l'interface Qt via les signaux et slots.

Semaine 15 - Semaine du 13 juin (*Livrable final*)

- Ajout des commentaires sur l'ensemble des fichiers et UML final mis à jour - 100%
- Rédaction du rapport final - 100%
- Enregistrement de la vidéo de démonstration - 100%

Le livrable final a été rendu avant le 13 juin 2026, conformément aux consignes du sujet.

Partie 5 - Les Contributions

Tableau récapitulatif

Membre	Spécialisation principale	Part de Contribution	Volume Horaire Estimé
Léonie	Gestion des tours, intégration globale, sauvegarde, mode solo	20.5%	52h
Lise	Ressources du jeu, plateaux, joueur, gestion des tours, UML	19.7%	50h
Estelle	Cartes Animal et Esprit, coordonnées hexagonales, motifs, commentaires	20.5%	52h
Emma	Interface graphique Qt complète, intégration moteur-vue, règles de placement	18.8%	48h
Basma	Calcul des scores, analyse du plateau, gestion de partie console, finalisation interface Qt	20.5%	52h
TOTAL	Projet complet	100%	254h

Léonie Leroux

Léonie a joué un rôle central dans le projet, en assurant à la fois ses tâches propres et un rôle d'intégratrice générale pour l'ensemble du groupe.

Dès le livrable 2, elle a développé la classe **CreationPartie** permettant d'initialiser une partie en console, mis en place le fichier **main.cpp** commun servant de menu de tests partagé entre tous les membres, et géré le diagramme UML global qu'elle a maintenu à jour tout au long du semestre (~13h).

Lors du livrable 3, elle a pris en charge la correction globale du projet (~12h) : réparation des headers manquants, correction des comparaisons invalides dans **PlaceVerif.cpp**, harmonisation des includes, suppression des définitions dupliquées, correction des crashes mémoire dans **PlateauJ** et correction de **CalculScore** pour relier les scores paysages et cartes. Elle a également fusionné l'ensemble des fichiers produits par les autres membres, notamment les cartes Animal et Esprit développées par Estelle, et ajouté un système de tests complet dans le main. Une première gestion des exceptions a été mise en place.

Durant la phase finale, Léonie a développé le système de sauvegarde complet (~27h) avec la classe **SauvegardePartie**, permettant d'enregistrer et de recharger l'intégralité d'une partie (pioche, plateau central, plateaux joueurs, cartes, cubes) dans un fichier texte. Elle a également réimplémenté le mode solo et intégré la distribution des cartes Esprit dans **GestionPartie**.

Lise Casanova Kinkela

Lise a posé les fondations du projet en développant l'ensemble des classes de ressources sur lesquelles tout le reste repose.

Dès le livrable 2 (~10h), elle a créé l'enum **CouleurJeton** et la classe **Jeton**, entièrement implémenté la classe **Place** représentant une case du plateau joueur, développé la classe **PlateauJ** avec gestion de la forme en quinconce du plateau A, avancé sur **PlateauC** et sur la classe **Pioche**.

Lors du livrable 3 (~12h), elle a corrigé la logique du plateau joueur pour respecter la forme réelle du plateau, créé la classe **Joueur** en la faisant hériter de **PlateauJ**, développé la classe **GestionTours** gérant le déroulement d'un tour de jeu, créé et adapté la classe **Cube** pour le lien avec les cartes Animal, et ajouté les tests correspondants dans le main.

Durant la phase finale, Lise a contribué à la correction des bugs d'intégration et a aidé à finaliser les interactions entre les modules qu'elle avait développés et le reste du projet. Elle a aussi repris l'UML pour avoir le rendu final. (~28h)

Estelle Pingard

Estelle s'est concentrée sur un périmètre bien défini mais techniquement exigeant : la gestion complète des cartes Animal et Esprit, et le système de coordonnées hexagonales permettant la vérification des motifs.

Lors du livrable 2 (~13h), elle a entièrement implémenté la structure **HexCoord** pour les coordonnées hexagonales cubiques et la gestion des rotations, et finalisé la classe **AnimalMotif** permettant de représenter et vérifier les motifs de placement des animaux. Elle a également avancé sur la classe **CarteAnimal**.

Lors du livrable 3 (~12h), elle a finalisé l'ensemble de la gestion des cartes : création de la classe de base **Carte**, développement de **CarteEsprit** avec la structure **EspritScoring**, gestion des pioches **PiocheAnimaux** et **PiocheEsprits**, création des 32 cartes Animal et des 10 cartes Esprit avec toutes leurs informations de jeu, uniformisation du code en séparant les fichiers d'en-tête et les fichiers sources, et mise à jour de l'UML.

Elle a également assuré tout au long du projet et en particulier sur phase finale une grande partie du travail de nettoyage du code et de commentaires, ainsi que l'affichage des motifs en mode console et a eu un rôle crucial lors de la phase de débogage. (~27h)

Emma Van Goethem

Emma a eu la responsabilité de l'intégralité de l'interface graphique Qt, qui constitue l'une des deux grandes exigences du sujet avec le mode console.

Lors du livrable 2 (~1h), elle a développé la classe **VérificationPlace** en soutien à Lise, et commencé à réfléchir à l'architecture de l'interface graphique.

Lors du livrable 3 (~5h), elle a conceptualisé les besoins visuels de l'interface, développé la fenêtre d'accueil et de configuration avec gestion dynamique du nombre de joueurs, et préparé l'intégration entre l'interface et le moteur de jeu.

Durant la phase finale (~42h), elle a développé l'intégralité de l'interface Qt : **MainWindow** avec détection automatique de sauvegarde, **SetupDialog** remplaçant **CreationPartie** en console, **GameWindow** orchestrant l'ensemble du déroulement du tour via signaux et slots, **PlateauCentralWidget** avec affichage des jetons colorés, **PlateauJoueurWidget** avec rendu hexagonal, **CartesAnimalWidget**, **ScoreWidget** et **ReglesDialog**. Elle a également corrigé les bugs liés à **Placeverif.cpp** et ajouté les méthodes d'accès nécessaires dans **GestionPartie** pour permettre à **GameWindow** d'interroger le moteur sans casser l'encapsulation.

Il est à noter que la charge de travail d'Emma s'est concentrée en grande partie sur la fin du semestre, l'interface graphique Qt n'intervenant qu'une fois le moteur console

suffisamment stable. Si cette organisation était logique d'un point de vue technique, elle a impliqué une période de travail intense sur les dernières semaines du projet, ce qui n'est pas idéal. Une meilleure anticipation de l'intégration entre le moteur et l'interface dès le milieu du semestre aurait permis de répartir cette charge plus équitablement dans le temps.

Basma Fengoura

Basma a rejoint le groupe lors du livrable 2 et a pris en charge le calcul du score, l'analyse du plateau et la gestion globale de la partie en mode console.

Lors du livrable 2 (~13h), elle a développé la classe **AnalysePlateau** permettant d'analyser le plateau joueur (voisins, groupes de cases connectées, cases vides) (80%), avancé sur **ScorePaysage** pour le calcul des points liés aux paysages (70%), et commencé **CalculScore** pour agréger les scores (70%). La classe **AideRegles** a également été initialisée.

Lors du livrable 3 (~15h), elle a finalisé le calcul du score et ajouté les tests correspondants, mis en place la gestion globale d'une partie dans **GestionPartie**, intégré les conditions de fin de partie et le calcul des scores finaux, ajouté l'annonce du gagnant, réorganisé les menus console et considérablement amélioré l'affichage général : affichage du plateau joueur en format court, informations pendant la partie, affichage détaillé des scores finaux. Elle a également effectué une vérification manuelle complète de la version console en solo et multijoueur.

Durant la phase finale (~24h), elle a contribué à la correction des derniers bugs d'intégration et aux tests finaux. Ainsi qu'à l'amélioration de l'interface pour avoir la version finale: affichage (carte, cube, plateau central (type A et B), l'intégration des illustrations, affichage de jetons sur le plateau à la place de lettres dans la version précédente.

Bien qu'arrivée en cours de projet lors du livrable 2, Basma a su s'intégrer rapidement au groupe et rattraper son retard. Son implication et ses compétences ont fait d'elle un membre essentiel du groupe, particulièrement sur la fin du projet.

Conclusion

Le développement de notre application **Harmonies** dans le cadre de l'**UV LO21** a été une **expérience enrichissante**, tant sur le plan technique que méthodologique. Nous avons appris à **travailler de manière collaborative** en gérant **un projet d'une envergure conséquente**, étalé sur l'ensemble du semestre, avec les défis que cela implique : répartition des tâches, intégration de modules développés en parallèle, et gestion des imprévus comme l'arrivée d'un nouveau membre en cours de route.

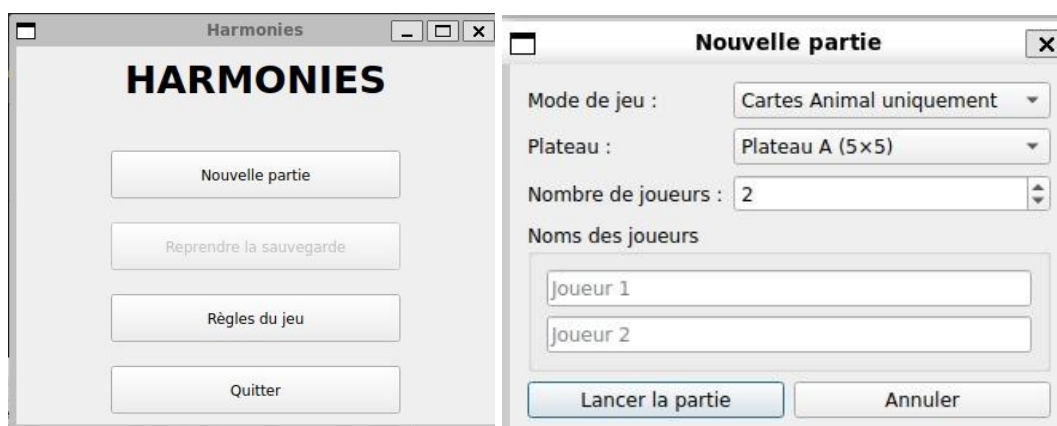
Sur le plan technique, **ce projet nous a permis de mettre en application les concepts de programmation orientée objet étudiés en cours** : hiérarchies de classes, héritage, encapsulation, et gestion manuelle de la mémoire en C++. Nous avons cherché à **concevoir une architecture modulaire et évolutive**, en séparant clairement le modèle de données, la logique de jeu et l'interface graphique. Le recours au framework Qt et à son mécanisme de signaux et slots nous a permis de réaliser **cette transition entre le mode console et le mode graphique de façon propre**, sans impacter le moteur de jeu.

Ce projet nous a également confrontés aux **réalités du développement en équipe** : la phase d'intégration s'est avérée plus complexe que prévu, et une part significative du temps a été consacrée à **stabiliser et corriger un code développé de façon trop cloisonnée**. C'est sans doute la leçon la plus importante que nous retenons : **l'importance de tester et d'intégrer régulièrement** plutôt que d'attendre que chaque module soit finalisé de son côté.

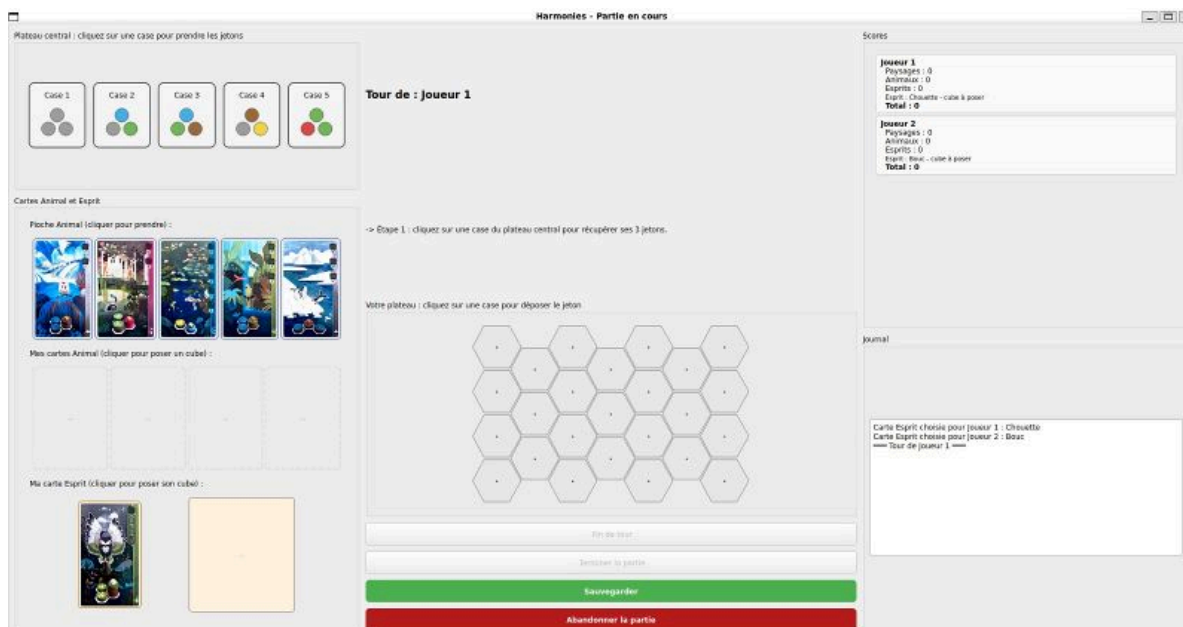
L'application **Harmonies** est aujourd'hui **pleinement fonctionnelle, stable, et respecte l'ensemble du cahier des charges initial**. Nous sommes fières du chemin parcouru depuis la première ébauche d'UML jusqu'à l'interface graphique finale.

Annexe

L'annexe ci-dessous présente quelques captures d'écran de l'application finale, illustrant les différentes fenêtres développées sous Qt. On y retrouve la fenêtre d'accueil avec le menu principal, la boîte de dialogue de configuration d'une nouvelle partie, le plateau central avec ses cases de jetons colorés, la fenêtre de choix de la carte Esprit, l'affichage des scores en temps réel, le module des règles par onglets ainsi que la zone des cartes Animal et Esprit. Ces captures donnent un aperçu concret du rendu visuel final de l'interface graphique.



Vue générale de la fenêtre de jeu (GameWindow) en cours de partie à deux joueurs, affichant le plateau central, les cartes, le plateau joueur hexagonal, les scores et le journal.



Scores

Joueur 1

Paysages : 0
Animaux : 0
Total : 0

Joueur 2

Paysages : 0
Animaux : 0
Total : 0

Règles du jeu - Harmonies

Jetons Paysages Cartes Animal Plateaux Déroulement du tour

Jeton bleu : se place uniquement au sol. Sur le plateau A, il sert surtout à former une rivière. Sur le plateau B, il sert à former des îles.

Jeton gris : peut être placé au sol, puis empilé uniquement sur du gris. Une montagne marque surtout si elle touche au moins une autre montagne.

Jeton marron : peut être placé au sol. Il peut aussi être empilé sur un autre marron pour préparer un grand arbre.

Jeton vert : peut être placé seul au sol, ou sur une base marron. Les arbres rapportent selon leur hauteur : 1, 3 ou 7 points.

Jeton jaune : se place uniquement au sol. Les champs rapportent par groupes d'au moins 2 cases jaunes adjacentes.

Jeton rouge : se place au deuxième niveau, sur un jeton gris, marron ou rouge. Il marque si les cases voisines montrent assez de couleurs différentes.

Fermer

Cartes Animal et Esprit

Pioche Animal (cliquer pour prendre) :



Mes cartes Animal (cliquer pour poser un cube) :

